

开发架构与框架技术概述—复习要点

1 课程概述与基本概念

1.1 软件开发架构平台

[p.1–4]

- > **课程定位:** 软件开发架构平台, 基于 Java EE 平台的企业级 Web 应用开发 [p.1]
- > **前置知识 (Prerequisites)** [p.2]: JavaSE、DB/SQL、JDBC、JSP/Servlet、HTML/CSS/XML、JavaScript、AJAX
- > 课程涵盖两大方向 [p.2]:
 - **开发架构 (Development Architecture):** 系统分层 MVC、前后端分离、各种框架技术
 - **系统架构 (System Architecture):** 数据缓存技术、服务器集群部署、服务和 Rest API 设计
- > **考核方式:** 闭卷考试 60% + 实验 40%, 实验分组 4-6 人 [p.4]
- > **参考书籍:** 《Spring in Action》 [p.5]

1.2 MVC 经典开发架构

[p.7–9]

- > 经典的 **MVC 开发架构 (Model-View-Controller)** 已成为 Java Web 开发的事实标准 [p.7]
- > MVC 中的 **Model** 层进一步细分为三层 [p.8]:
 - **Service:** 完成业务逻辑的业务类
 - **DAO (Data Access Object):** 实现数据持久化操作
 - **POJO (Plain Old Java Object):** 仅用于表达数据的值对象
- > 整个请求流程: 请求 → Servlet (Controller) → Service → DAO → POJO → 数据库 → JSP (View) → 响应 [p.8]
- > 严格遵循 MVC 的文件/包结构是项目规范的基础 [p.9]

2 框架技术的由来与发展

2.1 为什么需要框架?

[p.10, p.13–14]

- > MVC 开发中的两个核心问题 [p.10]:
 1. 如何约束程序员遵循 MVC 架构
 2. 如何简化和规范 MVC 架构下的代码开发
- > 对 Servlet 项目代码的分析发现 [p.13]:
 - **相同点:** 每个 Servlet 程序的流程基本一致——从客户端获取数据、调用业务逻辑处理、根据结果响应视图
 - **不同点:** 具体操作的数据 (个数、类型等)、调用的业务逻辑方法、响应的 JSP 页面
- > **框架的核心思想** [p.13]: 使用配置文件设定可变部分, 将不可变部分用框架方式固化, 不由程序员编写。这样既约束了 MVC 架构规范, 又提高了开发效率, 同时保证可维护性和可扩展性

2.2 三类框架 [p.14]

- > **表示层框架 (Presentation Layer Framework):** Struts、Spring MVC 一解决 View 层 (JSP) 和 Controller 层 (Servlet) 的规范和简化 [p.14]
- > **持久层框架 (Persistence Layer Framework):** MyBatis、Hibernate、JPA 一解决 Model 层中 DAO 数据访问对象代码的规范和简化 [p.14]
- > **容器类框架 (Container Framework):** Spring、EJB 一解决各组件间的耦合问题 (横向耦合和纵向耦合) [p.14]

2.3 主流框架组合演进 [p.15]

- > **SSH:** Spring + Struts2 + Hibernate [p.15]
- > **SSM:** Spring + Spring MVC + MyBatis [p.15]
- > **SSM + SpringBoot:** SpringBoot + Spring MVC + MyBatis (当前主流方案) [p.15]

2.4 框架的侵入性 (Invasiveness) [p.24]

- > **定义:** 使用第三方框架技术导致项目自身代码改变的程度 [p.24]
- > **高侵入性 (High Invasiveness) [p.24]:**
 - 直接继承或实现第三方框架的类或接口
 - 脱离框架项目无法运行, 重构和单元测试效率降低
 - 代表: Struts 1.x
- > **低侵入性 (Low Invasiveness) [p.24]:**
 - 通过反射、动态代理等语言特性, 结合 IoC 和 AOP 等架构理论, 动态调用框架接口
 - 脱离框架项目依然可运行, 可维护性和可扩展性高
 - 代表: Spring、MyBatis
- > 软件架构核心目标”高内聚、低耦合”(High Cohesion, Low Coupling) 的主要目的也是降低侵入性 [p.24]

3 表示层框架详解

3.1 Struts 1 框架 [p.17–23]

- > **历史 [p.17]:**
 - 2000 年 5 月 Craig R. McClanahan 提交 Web 框架原型
 - 2001 年 6 月 Struts 1.0 发布, 成为 ASF Jakarta 子项目
 - 2004 年 3 月成为 ASF 顶级项目
 - 2013 年 4 月 EOL (End of Life)
- > **基本原理 (五步配置) [p.18–23]:**
 1. 导入 Struts 1.x 对应的依赖包 [p.19]
 2. 配置 web.xml 接管请求 (**Servlet** 方式) [p.20]
 3. 创建 **ActionForm** 对象 (封装表单数据) [p.21]
 4. 创建 **Action** 对象 (处理业务逻辑, 需**继承框架基类**) [p.22]
 5. 完成 struts-config.xml 配置 (ActionForm 与 Action 的映射关系) [p.23]
- > **特点:** 高侵入性, Action 需继承框架类, 配置繁琐 [p.24]

3.2 Struts 2 框架

[p.17, p.25–30]

> 历史 [p.17]:

- 前身是 WebWork 框架，后拆分为 WebWork 2.0 和 XWork 1.0
- 2006 年合并改名为 Struts 2，本质上与 Struts 1.x 无关
- 最新版本 2.5.26，流行版 2.3.x
- 目前 Spring MVC 使用率已超过 Struts 2 [p.17]

> 核心改进思想：约定优于配置 (Convention over Configuration) [p.25]

> 基本原理（五步配置）[p.26–30]:

1. 导入 Struts 2.x 对应的依赖包 [p.26]
2. 配置 `web.xml`，由 **Filter** 接管（替代 Servlet 接管）[p.27]
3. 创建业务领域对象 **POJO** 类（普通的 Java 类，无需继承框架类）[p.28]
4. 创建 Action 对象——与 Java Web 容器**完全解耦**（低侵入性）[p.29]
5. 完成 `struts.xml` 配置 [p.30]

> Struts 2 相比 Struts 1 的最大进步：Action 不继承框架类，降低侵入性；Filter 替代 Servlet 接管 [p.27, p.29]

3.3 Spring MVC 框架基本原理

[p.31–35]

> 请求处理流程（七步）[p.31]:

1. 请求到达**前端控制器** (DispatcherServlet)，委托给具体控制器
2. 前端控制器通过查询**处理器映射** (Handler Mapping)，找到 URL 对应的控制器
3. 控制器处理请求（处理数据、调用业务逻辑）
4. 控制器将**模型数据** (Model) 和**逻辑视图名**返回给前端控制器
5. **视图解析器** (View Resolver) 将逻辑视图名匹配成具体的视图实现
6. 视图进行模型数据和视图实现的渲染
7. 交付模型数据，给出 Web 响应

> 配置要点 [p.32–35]:

- Maven 依赖：spring-webmvc [p.32]
- 配置 `web.xml` (DispatcherServlet 配置) [p.33]
- 配置 `applicationContext.xml` (Spring 上下文配置) [p.34]
- 编写 Controller [p.35]:
 - * `@Controller`: 声明该类为 Servlet 控制器
 - * `@GetMapping`: 注解 URL 映射
 - * `@ResponseBody`: 直接以字符串内容进行响应

4 三大表示层框架对比

特性	Struts 1 [p.18–23]	Struts 2 [p.25–30]	Spring MVC [p.31–35]
侵入性	高（继承基类 Action）	低（POJO Action）	低（注解驱动）

特性	Struts 1	Struts 2	Spring MVC
请求接管方式	Servlet	Filter	Servlet (DispatcherServlet)
配置文件	struts-config.xml	struts.xml	web.xml + applicationContext.xml
核心改进	首次引入 MVC 框架	约定优于配置, Filter 接管	注解驱动, REST 支持
与 Spring 集成	需额外配置	需额外配置	原生集成
当前状态	EOL (2013 年)	使用率下降	主流首选

5 Maven 项目构建管理工具

5.1 Maven 简介 [p.37]

- > Maven 是基于 项目对象模型 (POM, Project Object Model) 的项目管理机制 [p.37]
- > 两大核心功能 [p.37]:
 - 1. 通过 pom.xml 管理项目内部模块间和外部插件的依赖关系
 - 2. 实现项目的自动化构建 (Build) 和部署运行

5.2 Maven 仓库体系 (Repository) [p.38–39]

- > 三种仓库类型 [p.38]:
 - 本地仓库 (Local Repository): 本机存储已下载的依赖
 - 中央仓库 (Central Repository): Maven 官方维护的开源项目依赖, <https://mvnrepository.com/>
 - 远程仓库/私服 (Remote Repository / Private Server): 组织或公司内部搭建的第三方仓库
- > 依赖搜索顺序 [p.39]: 本地仓库 → 私服 → 中央仓库 → 远程仓库
- > 私服作用: 统一依赖版本、解决网络问题, 可在 settings.xml 中配置 [p.39]

5.3 POM 核心概念 [p.40]

- > 每个 Maven 项目有唯一的 pom.xml 文件 [p.40]
- > 坐标 (Coordinate) 三要素 [p.40]:
 - groupId: 组织或项目组标识
 - artifactId: 模块名称
 - version: 版本号
- > 依赖声明: 通过 <dependencies> 节点声明, 每个 <dependency> 表示一种依赖, 也通过坐标三要素指定 [p.40]

5.4 Maven 约定目录结构 [p.41]

- > src/main/java: 项目 Java 源代码
- > src/main/resources: 资源文件 [p.41]
- > src/test/java: 测试源代码 [p.41]

- > **src/main/web**: Web 项目相关文件（HTML、CSS、JavaScript 等） [p.41]
- > **target**: 编译结果存放
- > **pom.xml**: 位于项目根目录下 [p.41]

5.5 Maven 生命周期命令

[p.42]

命令	作用	说明
mvn compile	编译	将 src/main/java 编译成 class 到 target 目录
mvn test	测试	编译并运行 src/test/java 中的测试代码
mvn clean	清理	删除 target 目录
mvn package	打包	生成.jar 或.war 打包文件
mvn install	安装	将打包文件上传到本地仓库
mvn deploy	部署/发布	将打包文件上传到 Web 服务器或私服

5.6 主流自动化构建工具

[p.43]

- > **Maven**: 基于 XML 配置，依赖管理强大，使用最广泛
- > **Gradle**: 基于 Groovy DSL，灵活性高，性能更好
- > **Ant**: 最早的 Java 构建工具，过程式定义，灵活性高但配置繁琐

6 本章小结与考点梳理

6.1 核心知识体系

[p.1–44]

- > **MVC 架构** [p.7–9]: 三层细分（Service / DAO / POJO），请求处理流程
- > **三类框架** [p.14]: 表示层框架、持久层框架、容器类框架——各自解决的问题
- > **框架侵入性** [p.24]: 定义、高侵入性 vs 低侵入性的区别和代表框架，”高内聚低耦合”目标
- > **框架组合演进** [p.15]: SSH → SSM → SpringBoot+SSM
- > **表示层框架对比** [p.17–35]: Struts 1（高侵入/Servlet 接管）→ Struts 2（低侵入/Filter 接管/约定优于配置）→ Spring MVC（注解驱动/七步请求流程/DispatcherServlet）
- > **Maven** [p.37–43]: POM 机制、仓库体系（本地/中央/私服）、依赖搜索顺序、坐标三要素、约定目录结构、六条核心命令

6.2 考试重点提示

高频考点: (1) MVC 三层细分及各层职责 [p.8]; (2) 三类框架分别解决什么问题 [p.14]; (3) 框架侵入性的概念与高低对比 [p.24]; (4) Spring MVC 七步请求处理流程 [p.31]; (5) Maven 坐标三要素与六条命令 [p.40, p.42]; (6) Struts 1 → Struts 2 → Spring MVC 的演进逻辑 [p.15, p.17]; (7) 约定优于配置的含义 [p.25]